

Module 9) Python DB and Framework

1. HTML in Python

Theory:

□ Introduction to embedding HTML within Python using web frameworks like Django or Flask.

Web frameworks such as **Django** and **Flask** allow developers to build web applications using Python. These frameworks connect **Python code with HTML pages** to create dynamic websites. When a user sends a request through the browser, the server processes it using Python and returns an HTML page as a response. This integration helps in creating interactive and dynamic web pages

□ Generating dynamic HTML content using Django templates.

In **Django**, the **template system** is used to generate dynamic HTML content. Data from Python views is passed to HTML templates and displayed using template variables.

2. CSS in Python

Theory:

□ Integrating CSS with Django templates.

In **Django**, CSS is used to style HTML pages in templates. CSS files are linked to Django templates so that web pages can have proper design, colors, layouts, and fonts.

The CSS file is usually stored in a **static folder**, and then it is connected to the H

□ How to serve static files (like CSS, JavaScript) in Django.

- Create a **static folder** inside the project or app.
- Place files like CSS, JavaScript, and images inside this folder.
- Add the static file configuration in **settings.py**.
- Use {% load static %} in templates to access static files.

3. JavaScript with Python

Theory:

□ Using JavaScript for client-side interactivity in Django templates.

JavaScript is used in Django templates to add **client-side interactivity** to web pages. It allows actions such as form validation, button clicks, animations, and dynamic content updates without reloading the page.

In **Django** templates, JavaScript can be written directly inside the HTML file or included from external files.

□ Linking external or internal JavaScript files in Django.

1. Internal JavaScript:

JavaScript code is written directly inside the HTML template using the `<script>` tag.

Example:

```
<script>
console.log("Welcome to Django website");
</script>
```

2. External JavaScript:

JavaScript code is stored in a separate .js file inside the **static folder** and linked in the template.

Example:

```
{% load static %}
<script src="{% static 'js/script.js' %}"></script>
```

4. Django Introduction

Theory:

□ Overview of Django: Web development framework.

Django is a high-level web development framework written in Python. It is used to build secure, scalable, and dynamic websites quickly. Django follows the **Model–View–Template (MVT)** architecture, which separates application logic, user interface, and data management. It also provides built-in features such as authentication, database management, and an admin panel.

□ Advantages of Django (e.g., scalability, security).

- **Scalability:** Django can handle large applications and heavy traffic efficiently.
- **Security:** It provides built-in protection against common attacks like SQL injection and cross-site scripting.
- **Rapid Development:** Many built-in tools help developers build applications faster.
- **Built-in Admin Panel:** It provides an automatic admin interface for managing data.
- **Reusable Components:** Developers can reuse modules and applications easily.

□ Django vs. Flask comparison: Which to choose and why.

Feature	Django	Flask
Type	Full-featured framework	Micro framework
Complexity	More features and structure	Simple and lightweight
Built-in Tools	Authentication, ORM, Admin panel	Minimal built-in features
Best For	Large and complex web applications	Small or simple projects

5. Virtual Environment

Theory:

□ Understanding the importance of a virtual environment in Python projects.

A **virtual environment** in Python is an isolated environment that allows developers to install project-specific libraries without affecting other projects. It helps manage different package versions for different projects.

Using a virtual environment ensures that dependencies remain organized and prevents conflicts between packages. It is commonly used in Python web development frameworks like **Django**.

□ Using venv or virtualenv to create isolated environments.

Using venv or virtualenv to Create Isolated Environments

Python provides tools like **venv** or **virtualenv** to create virtual environments.

Steps:

1. Create a virtual environment

```
python -m venv myenv
```

2. Activate the environment

- a. Windows:

```
myenv\Scripts\activate
```

- b. Mac/Linux:

```
source myenv/bin/activate
```

3. Install required packages inside the environment using pip.

This keeps the project dependencies separate from the global Python installation.

6. Project and App Creation

Theory:

□ Steps to create a Django project and individual apps within the project.

1. Install Django

```
pip install django
```

2. Create a Django project

```
django-admin startproject myproject
```

3. Move into the project folder

```
cd myproject
```

4. Create a Django app

```
python manage.py startapp myapp
```

7. MVT Pattern Architecture

Theory:

□ Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

Django follows the **Model–View–Template (MVT)** architecture. It separates the application into three main components to make development organized and efficient.

Components of MVT:

- **Model:** Manages the database and defines the structure of data.
- **View:** Contains the business logic and processes user requests.
- **Template:** Handles the presentation layer and displays data in HTML format.

Request–Response Cycle:

1. The user sends a request through a web browser.
2. The request first goes to **urls.py**, which maps the URL to a specific view.
3. The **view** processes the request and interacts with the **model** if database data is needed.
4. The view sends data to the **template**.
5. The template generates the HTML page and sends the **response back to the browser**.

8. Django Admin Panel

Theory:

❑ Introduction to Django's built-in admin panel.

Django provides a **built-in admin panel** that allows developers to manage website data easily. It automatically generates an administrative interface for managing database records such as adding, editing,

❑ Customizing the Django admin interface to manage database records.

```
from django.contrib import admin
from .models import Product
```

```
admin.site.register(Product)
```

9. URL Patterns and Template Integration

Theory:

❑ Setting up URL patterns in `urls.py` for routing requests to views.

In **Django**, the **`urls.py`** file is used to define URL patterns that route user requests to the appropriate views. When a user enters a URL in the browser, Django checks the **`urls.py`** file to determine which view function should handle the req

❑ Integrating templates with views to render dynamic HTML content.

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

10. Form Validation using JavaScript

Theory:

❑ Using JavaScript for front-end form validation.

JavaScript is used for **front-end form validation** to check user input in the browser before the data is sent to the server. It helps ensure that users enter correct information such as required fields, valid email addresses, or proper password formats.

Form validation improves **user experience** and reduces unnecessary server requests by detecting errors immediately.

```
<form onsubmit="return validateForm()">
```

```
  Name: <input type="text" id="name">
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

```
<script>
```

```
function validateForm() {
```

```
  var name = document.getElementById("name").value;
```

```
  if (name == "") {
```

```
    alert("Name must be filled out");
```

```
    return false;
```

```
  }
```

```
}
```

```
</script>
```

11. Django Database Connectivity (MySQL or SQLite)

Theory:

□ Connecting Django to a database (SQLite or MySQL).

Django supports different databases such as **SQLite** and **MySQL** for storing and managing application data.

By default, Django uses **SQLite**, which is a lightweight database suitable for development and small projects. For larger applications, databases like **MySQL** can also be used.

Steps to Connect Django with a Database

1. Open the **settings.py** file in the Django project.
2. Configure the **DATABASES** section with the required database details.

Sql:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'mydatabase',  
        'USER': 'root',  
        'PASSWORD': 'password',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

12. ORM and QuerySets

Theory:

□ Understanding Django's ORM and how QuerySets are used to interact with the database.

Django uses an **ORM (Object Relational Mapper)** to interact with the database using Python code instead of SQL queries. It converts database tables into Python classes called **models**.

A **QuerySet** is used to retrieve and manipulate data from the database.

Example:

```
Product.objects.all()  
Product.objects.filter(price=1000)
```

13. Django Forms and Authentication

Theory:

□ Using Django's built-in form handling.

Django provides a built-in **form handling system** that helps developers create, process, and validate forms easily. It automatically handles form data, validation, and error messages.

Developers create forms using **forms.py**, display them in templates, and process the submitted data in views.

Example:

14. CRUD Operations using AJAX

Theory:

❑ Using AJAX for making asynchronous requests to the server without reloading the page.

AJAX is used to send and receive data from the server **without reloading the web page**. It allows web applications to update parts of a page dynamically, improving user experience.

In **Django**, AJAX works with **JavaScript** to send requests to the server and receive responses asynchronously.

Example:

```
fetch('/data/')  
  .then(response => response.json())  
  .then(data => console.log(data));
```

15. Customizing the Django Admin Panel

Theory:

❑ Techniques for customizing the Django admin panel.

In **Django**, the admin panel can be customized to manage database records more efficiently. Developers modify the **admin.py** file to control how data appears and behaves in the admin interface.

Common Techniques:

- **Registering Models:** Allows models to appear in the admin panel.
- **List Display:** Shows selected fields of a model in the admin list view.
- **Search Fields:** Enables searching of records in the admin panel.
- **Filters:** Allows filtering of records based on specific fields.

Example:

```
from django.contrib import admin  
from .models import Product  
  
class ProductAdmin(admin.ModelAdmin):
```

```
list_display = ('name', 'price')
search_fields = ('name',)
```

```
admin.site.register(Product, ProductAdmin)
```

16. Payment Integration Using Paytm

Theory:

□ Introduction to integrating payment gateways (like Paytm) in Django projects.

A **payment gateway** allows websites to accept online payments from users. In **Django** projects, payment gateways can be integrated to process transactions securely.

Services like **Paytm** provide APIs that connect the Django application with their payment system. When a user makes a payment, the request is sent to the payment gateway, which processes the transaction and returns the payment status to the website.

Basic Steps:

1. Create a merchant account with the payment gateway.
2. Obtain API keys or merchant credentials.
3. Integrate the gateway API into the Django project.
4. Redirect users to the payment page and handle the payment response.

This integration allows websites to accept **secure online payments** such as credit cards, debit cards, and digital wallets

17. GitHub Project Deployment

Theory:

□ Steps to push a Django project to GitHub.

- **Initialize a Git repository**

`git init`

- **Add project files**

`git add .`

- **Commit the files**

`git commit -m "Initial commit"`

- **Create a repository on GitHub** and copy the repository URL.
- **Connect local project to GitHub**

`git remote add origin repository_URL`

- **Push the project to GitHub**

`git push -u origin main`

18. Live Project Deployment (PythonAnywhere)

Theory:

□ Introduction to deploying Django projects to live servers like PythonAnywhere.

Deployment means making a web application available on the internet so users can access it online. A **Django** project can be deployed on live servers to run continuously and serve users.

Platforms like **PythonAnywhere** allow developers to easily host and run Django applications on the web.

Basic Steps for Deployment:

1. Create an account on the hosting platform.
2. Upload the Django project files to the server.

3. Configure the virtual environment and install required packages.
4. Set up the database and static files.
5. Run the web application so it becomes accessible online.

After deployment, users can access the Django website using a public URL on the internet.

Theory:

□ Setting up social login options (Google, Facebook, GitHub) in Django using OAuth2.

Social login allows users to sign in to a **Django** website using accounts from platforms like **Google**, **Facebook**, or **GitHub**. This is implemented using **OAuth 2.0**, which provides secure authentication without sharing user passwords.

Steps:

1. Register the application on the provider (Google, Facebook, or GitHub).
2. Obtain the **Client ID** and **Client Secret**.
3. Install a Django package such as **django-allauth**.
4. Configure authentication settings in Django.
5. Add social login buttons in the website.

20. Google Maps API

Theory:

□ Integrating Google Maps API into Django projects.

The **Google Maps API** can be integrated into **Django** projects to display maps, locations, and directions on a website. It allows developers to show interactive maps and location-based services.

Steps:

1. Create a project in the **Google Cloud Platform**.
2. Enable the **Google Maps API** and generate an API key.
3. Add the API script in the Django HTML template.
4. Use JavaScript to display the map on the webpage.

Example:

```
<div id="map" style="height:400px;"></div>
```

```
<script  
src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY"></script>
```

```
<script>
```

```
function initMap() {
```

```
    var location = {lat: 23.0225, lng: 72.5714};
```

```
    var map = new google.maps.Map(document.getElementById("map"), {
```

```
        zoom: 10,
```

```
        center: location
```

```
    });
```

```
}
```

```
</script>
```